No.

2-dars: Model Improvement.

Tasavvur qil, maktabda 10 ta odam bor.

Haqiqiy holat

Odam	Kasalmi?	
A	✓	5 ta kasal
B	✓	
C	✓	
D	✓	
E	✓	
F	✗	5 ta kasal emas
G	✗	
H	✗	
I	✗	
J	✗	

10 ta odam

Endi model aytadi: "A, B, C, F kasal"
Demak model: A to'g'ri, B to'g'ri, C to'g'ri F no'to'g'ri (aslida so'g'lom)

Shunda: TP = 3 → to'g'ri topilgan kasallar
FP = 1 → bekordan-bekor kasal d-di.
FN = 2 → kasal edi lekin topolmadi (D va E)

Endi Precision
Savol: "Model kasal degan larning nechta qismi rostdan kasal?"

Model 4 kishini kasal dedi: A, B, C, F
lekin F yolg'on.
Demak 4tadan 3tasi to'g'ri. Precision = $\frac{3}{4}$ = 75%

Endi Recall
Savol boshqa: "Aslida kasal bo'lganlarning nechta qismini topdi?"
Aslida 5 ta kasal bor edi: A, B, C, D, E
Model faqat 3 tasini topdi. Recall = $\frac{3}{5}$ = 60%

Eng katta farq
→ Precision qarayadi: "Men topganlarim ichida nechta to'g'ri?"
→ Recall qarayadi: "Haqiqiy = nechta qismin ~~to'g'ri~~ topdim

Juda qisqa intuition: Tasavvur qil politiya jinoyatchi qidirayapti.

High Precision
Politsiya faqat aniq ushlaydigan odam/ni ushlaydi.
→ Kam odam ushlaydi
→ lekin ushlaganlarni ko'pincha to'g'ri: precision ↗

High Recall
Politsiya hammani ushlaydi 🎃
→ Jinoyatchilar deyarli qochmaydi
→ lekin ko'p begunohlar ham tushadi.
Recall ↗ ammo Precision ↘

Model Improvement::
- Data Based
  - ↳ Advanced Missing Value Treatment
- Method based
- Algorithms Based
  - ↳ Precision - Recall trade - offs
- Tuning Based

Advanced Missing Value Treatment

Tushurib qoldirilgan qiymatlar cell ko'rinishida yani bosh yoki N/A ko'rinishida bo'lishi mumkin.

- Missing qiymatlar faqatgina bosh katak emas!
- Missing qiymatlar:
  - ↳ Modelni o'zgartiradi
  - ↳ Bias (model = noto'g'ri assumptions sababli yuzaga keladigan sistematik xato) (tahmin → faraz)
  - ↳ Noto'g'ri natija qiladi
  - ↳ Mantiqsiz natija qiladi
  - ↳ Stabil bolmagan model

Ovqat qilganimizda tuz qolib ketsa, qolgan masalliqlar yaxshi solganimiz bn mazali bolmasligi mumkin. Endi, o'ylaypmizda, nimadir qolib ketdi qeyvorpmiz, uni aynan nimaligini yaxshiroq tushunmasdan turib, tuz qo'shib qo'yak, o'zim tuz qo'shgan bolishimiz mumkin, lekin bizga aslida qulampir kerak bolgandir boshqa ziravor kerak bolgandir. Unda baribir buziladi yana, noto'g'ri qiymat bilan to'ldirib qo'yganimiz u-n.

No.

- Imputation == Process filling missing values

Buni 2-qismga bolsak boladi: → df.describe() yilsak shunday info olishimiz mumkin.

1. Basic statistical methods like mean, median, mode, fixed value, drop rows
2. Advanced.

↳ Types of Missing Value Imputation (Advanced Methods)

- Basicdan kora olgilli va yaxshiroq ishlaydigan usullar
↳ Bunda toldirish boyicha ham manoli yondashamiz. Ya'ni oldingi method larimizdan aqilliroq bolgan usul bilan toldiramiz

- Ular qatlar orasidagi (pattern) orasidagi aloqadan kelib chiqqan holatda munosabatdan organadi

I. Mathematical Statistical Imputation (Advanced)
↳ KNN (Similarity-Based Imputation)
↳ MICE (Multiple Imputation By Chained Equations)

II. Machine Learning Methods
↳ Predictive (Model-Based) Imputation => (Tushirib qoldirilgan qiymatlarni predict qilamiz) => ya'ni tushib qolgan qiymat qayisi bolishi mumkin deb predict qilamiz.

III. Rule-Based / Domain Knowledge Methods (Soxa bilmidan kelib chiqib toldirish)
↳ Domain-Aware Imputation

No.

Encoding shart

IV. Implementation Strategy

↳ Imputation Inside Cross-Validation (Leakage Prevention) => Qiymat/ni o'zib ketishini oldini olish. yoki sizib chiqishini oldini oladi. (Model training paytida bilmasligi kerak bolgan ma'lumotni bilib qoladi)

- KNN Imputer (Similarity-Based Imputation)

↳ KNN imputer feature larga kora eng o'xshash k ta qiymat/ni topadi va ularning qiymat/i o'rtacha tushirib qoldirilgan qatorlarni to'ldiradi.

- Qachon:
  - O'xshash featurelar bolganda
  - O'xshash qiymat/ bolganda
  - Mean va Mediandan kora yaxshiroq
- Maktabda o'quvchi boyini yozishni unutib qoldirdi. 5 ta eng yaqin do'stini (bo'y, yosh va jinsida) dan kelib chiqib dostarini bo'yi bilan to'ldiramiz.

↳ KNNdan foydalanishdan oldin Encoding qilib olish k-k.

```
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=5)
df_imputed = imputer.fit_transform(df)
df_imputed = pd.DataFrame(df_imputed, columns=df.columns)
df_imputed.isnull().sum()
```

↳ Bu xolatda array(C..I) ko'rinishida bolib qoladi. Biz buni dataset ko'rinishiga o'tkazib olamiz undan pastdagi kod orqali.

No.

Encoding shart

- MICE (Multiple Imputation by Chained Equations)
  ↳ MICE usuli tushurib qoldirilgan qiymatli bor ustun bir feature u-n alohida model qurish orqali u/ni to'ldiradi.
  ↳ Ustun bir model cycle (loop) ko'rinishida run bo'ladi (bir chekkadan)
  ↳ 10 ta shifokor ko'rigida → xar xil javob → umumlashtirish → yaxshiroq natija.

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.ensemble import RandomForestRegressor

imputer = IterativeImputer(estimator=RandomForestRegressor,
                           max_iter=10)
df_imputed = imputer.fit_transform(df)
df_imputed = pd.DataFrame(df_imputed, columns=df.
                          columns)
```

Encoding shart

- ML methods: Predictive (Model-Based) Imputation
  ↳ Tushirib qoldirilmagan feature lar orqali tushirib qoldirilgan qiymatlarni topish va ularni toldirish u-n model train qilamiz.
- Algorithms:
  ↳ Random Forest
  ↳ XGBoost
  ↳ Linear Regression
- Target qiymatni predict qilish bilan bir xil usul

No.

```
from sklearn.ensemble import RandomForestRegressor

for col in df.columns:

    if df[col].isnull().sum() == 0:
        continue

    train = df[df[col].notnull()]
    test  = df[df[col].isnull()]

    X_train = train.drop(col, axis=1)
    y_train = train[col]

    X_test = test.drop(col, axis=1)

    model = RandomForestRegressor()
    model.fit(X_train, y_train)

    # Predicted missing values
    df.loc[df[col].isnull(), col] = model.predict(X_test)
```

joylash

- Rule - Based / Domain Knowledge Methods:
  Domain - Aware Imputation.
  - ↳ Not math = use domain knowledge
  - ↳ Missing salary → use (league average × position ratio)
  - ↳ Missing xG → use (shots × 0.1)

No.

↳ Missing position_ratio → Based on position:

- FW = 4
- CM = 6
- CB = 4

Buni ancha ishi bor ekan!

- Implementation Strategy:
  ↳ Imputation inside cross-validation (leakage prevention)
  ↳ Butun datasetni impute qilmasdan xar bir qismini alohida-alohida impute qilish.

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score
from sklearn.impute import KNNImputer
from sklearn.ensemble import RandomForest~~Classifier~~Regressor
from sklearn.model_selection import train_test_split

X = df.drop('Churn', axis=1)
y = df['Churn']

X_train, X_test, y_train, y_test = train_test_split(
X, y, train_size=0.2, random_state=42)

pipeline = Pipeline([
    ("impute", KNNImputer(n_neighbors=5)),
    ("model", RandomForestRegressor())
])
scores = cross_val_score(pipeline, X, y, cv=5)
print(scores)
```

No.

Missing Value Imputation – Qachon ishlatiladi + Afzalliklari / Kamchiliklari

1. KNN Imputer

Key idea

"Missing qiymatni eng o'xshash data point/ (nearest neighbors) yordamida to'ldiradi.

Yani: • O'xshash odamlni topadi, • Ularning qiymatlariga qarab missing valuelarni hisoblaydi

Example:

Age	Sallary
25	3000
26	3200
?	3100

Model aytadi: "3100 salaryli odamlar 25-26 yosh ekan."
Shuning uchun missing age ≈ 25 yoki 26 bolishi mumkin.

Advantages

- nonlinear relationshiplarni tushuna oladi.
- meandan agilliroq
- assumption kam

Disadvantages

- katta datasetda sekin
- scaling muhim
- categorical featurelarni encoding qilish k-k.

When we used?

Data ichida o'xshash patternlar bolsa.

No.

2. MICE (Multiple Imputation by Chained Equations)

Key idea

"Har bir missing columni boshqa featurelar yordamida qayta-qayta predict qiladi." -> Iterative ishlaydi.

Example:

Age	Salary	Experience
?	7000	12

MICE: "Experience va salarydan foydalanib ageni tasmin qiladi"

Keyin boshqa missinglarni ham shu tarzda iterative to'ldiradi

Advantages:

- Kuchli method
- Biasni kamaytiradi
- Uncertainityni hisobga oladi.

Disadvantages

- Sekin
- Murakkab
- Unstable bo'lishi mumkin.

When we used

- Multiple columns'da missing bo'lsa
- featurelar orasida kuchli relationship bo'lsa.

3. Predictive (Model-Based) Imputation

Key idea

No.

" Missing qiymatni predict qilish u-n alohida ML model train qiladi "

Example: train data ↓

Age	Salary	Experience
25	3000	2
40	8000	15

Model Relationship organadi

Kegin:

Age	Salary	Experience
?	7000	12

Model Predict qiladi: Age ≈ 37

Advantages
- yuqori accuracy bolishi ~~u-n~~ mumkin
- murakkab relationship/ni ushlaydi
-

Disadvantages
- leakage riski bor
- sekinroq
- encoding kerak

When we used
- ML prediction task'larida
- feature relationship'lari kuchli bolsa

No.

4. Domain - Aware Imputation

Key Idea

"Sohaga oid bilim (domain knowledge) orqali missing value to'ldiriladi"

Example:

- Football dataset: Goalkeeper → goals odatda past bo'ladi
- Medical dataset: newborn baby → salary = 0 bo'lishi mumkin.

Advantages

- Simple
- Interpretable
- Leakage kam

Disadvantages

- human knowledge'ga bog'liq
- accuracy past bo'lishi mumkin.

When we used

Domain Patternlar aniq bo'lsa

5. Imputation Inside Cross-Validation

key idea: "leakage oldini olish u-n har fold ichida alohida imputation qilinadi"

Wrong ✗

Barcha datada Imputation: imputer.fit(all_data)
bu leakage bo'lishi mumkin.

No.

Correct ✓

faqat train foldda: imputer.fit(X_train) keyin imputer.fit_transform(X_test)

Advantages:

- leakage'ni oldini oladi
- model yaxshi generalize qiladi

Disadvantages

- implemantation murakkabroq (uni kodda ishlatish murakkabroq)

When we used

har doim ML pipeline'da